WeatherGo System Architecture

BACKEND

1. Data Collection Layer

Purpose: Collects up-to-date meteorological data from public APIs on a scheduled basis.

Components:

- Amazon EventBridge – a scheduler that triggers the data collection process every few hours.
- AWS Lambda “data_collector” (alternative: AWS Fargate Task) – calls APIs such as Open-Meteo or OpenWeather, retrieves key parameters (temperature, humidity, wind_speed, pressure, cloud_cover) and saves them to storage.
- Amazon S3 (/raw/) – stores raw JSON weather data files.

2. Data Processing Layer

Purpose: Cleans and normalizes raw data into structured datasets.

Components:

- AWS Lambda “data_cleaner” (alternative: AWS Glue Job) – reads files from /raw/, removes missing values, aggregates by time, and saves the results as CSV or Parquet files.
- Amazon S3 (/processed/) – stores cleaned and structured data ready for prediction.
- AWS Glue Data Catalog (optional) – maintains metadata about the data structure for SQL queries via Athena.

3. Model and Prediction Layer

Purpose: Generates weather predictions using a pre-trained ML model.

Components:

- Amazon S3 (/models/) – stores one or more pre-trained models, e.g., model.pkl.
- AWS Lambda “predict_weather” (alternative: Amazon SageMaker Endpoint or ECS Service) – loads the model from S3, reads the latest processed data,

computes predictions for given coordinates or time ranges, and outputs the results.

- Amazon S3 (/predictions/) – keeps prediction outputs in JSON or other format.

4. API and Integration Layer

Purpose: Connects the prediction system with user-facing services through a public API.

Components:

- Amazon API Gateway – exposes a REST API endpoint /forecast for integration with the Telegram bot or other applications.
- AWS Lambda “api_handler” (alternative: AWS Fargate API or Elastic Beanstalk with FastAPI) – handles HTTP requests, reads user parameters (coordinates, date, radius), fetches predictions from S3 or triggers predict_weather, and returns the response in JSON.
- Amazon CloudWatch Logs – monitors API traffic and execution performance.

5. Analytics and Monitoring Layer

Purpose: Supports data analysis, model accuracy evaluation, and operational stability.

Components:

- Amazon Athena – enables SQL-based analysis of datasets in S3 (processed/ and predictions/), such as accuracy or error metrics.
- Amazon CloudWatch – monitors Lambda performance, tracks execution times, and logs errors.
- AWS IAM – manages permissions for communication between AWS services (e.g., Lambda access to S3).

FRONTEND

1. Telegram Bot Application

Purpose: Provides a conversational interface for users to request weather forecasts in natural language.

Components:

- Telegram Bot (Python, python-telegram-bot library) – receives user messages such as:
  "I want to go on a date in Lviv on November 2 at 7 PM, within a 3 km radius, expecting sunny weather."
- The bot extracts parameters: location, date, time, radius, and expected_condition.
- It then sends a structured request to the backend API /forecast.

2. Telegram Webhook and API Integration

Purpose: Handles user requests, calls the backend, and returns predictions.

Components:

- AWS Lambda "bot_handler" (alternative: ECS Fargate container or FastAPI endpoint) – receives webhook events from Telegram, calls the API Gateway /forecast, retrieves the prediction, and formats the response message.
- Amazon API Gateway – acts as a communication layer between Telegram and backend infrastructure.
- Amazon S3 (/predictions/) – provides the stored forecast data used to generate responses.